B. Pivoting of SLUBStick

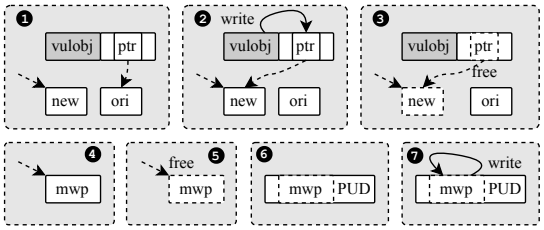
Figure 12 delineates SLUBStick's exploitation workflows. The **pivot by pointer** sequence comprises: ❶ allocating pointer-containing object ptr after vulnerable object $vulnobj$; ❷ triggering the vulnerability to redirect ptr's pointer to object new; ❸ releasing ptr's contained pointer, thereby freeing new; ❹ placing memory-write-primitive object mwp in new's freed slot; ❺ triggering new's inherent release to free mwp (pivot achieved), concurrently releasing other objects in mwp's memory page, enabling page recycling.

The **pivot by reference counter** proceeds via: ❶ allocating object ref after $vulnobj$ with refcount=2; ❷ corrupting ref's refcount to 1 via vulnerability; ❸ unreferencing ref to reduce refcount=0, forcing release; ❹ placing mwp in ref's freed slot; ❺ re-corrupting ref's refcount to 1; ❻ final unreferencing ref that frees mwp (pivot achieved), subsequent releases of other objects on the page enable recycling.

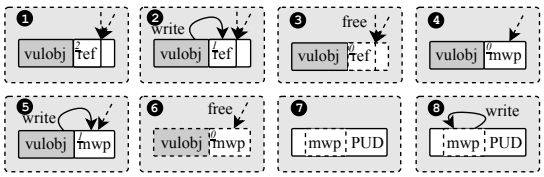
In the CVE-2022-0995 case study, SLUBStick employs **pivot by reference counter** against `watch_filter` as $vulnobj$ and `anon_vma` as ref. After establishing mwp (simulated via custom allocator since SLUBStick lacks native compatible objects): ❼ reclaiming mwp's page as PUD page; ❽ triggering mwp's write operation to corrupt PUD entries, illegally mapping large physical memory regions to user space - achieving arbitrary memory writes through page table manipulation.

C. Analysis of Time Window

Figure 13 establishes the correspondence between abbreviated object labels in Figure 12 and their temporal lifecycles. We model object operations – allocation, write, and recycle & reclaim (RR) – as occurring at equal time intervals. The horizontal axis represents temporal progression where overlapping rectangles indicate the number of concurrently active objects required at each timestamp. The vertical axis quantifies object lifetime durations through rectangle spans. Dark-shaded regions identify critical phases requiring simultaneous maintenance of multiple live objects, with increased shading density directly correlating to exploitation difficulty: each additional overlapping dark region imposes exponentially stricter constraints on timing window construction.



(a) pivot by pointer



(b) pivot by reference counter

Figure 12: Workflow of SLUBStick.

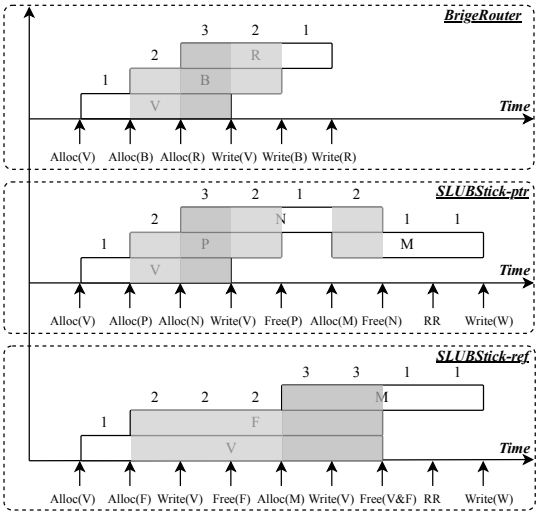


Figure 13: Lifespan of critical objects during exploitatoin. V represents the vulnerable object (`vulnobj`), B represents the `bridge` object, R represents the `router` object, P represents the `ptr` object , N represents the `new` object, F represents the `ref` object, and M represents the `mwp` object.